

# Tratamento de exceções

*Bib: Java: An Introduction to Problem Solving & Programming, Walter Savitch*

*Cap. 9: Exception Handling*

## Programação III (PG3)

João Pedro Patriarca, ([joao.patriarca@isel.pt](mailto:joao.patriarca@isel.pt)),

Gabinete F.0.23 do edifício F, LEETC, ADEETC



# Exceções – o que é?

---

- Um programa tem um fio de execução que corresponde, por norma, ao processamento em condições normais
  - O utilizador introduz sempre um inteiro quando lhe é pedido um inteiro
  - Uma coleção não está vazia quando há o acesso a uma entrada
  - Um ficheiro tem os dados pretendidos e devidamente formatados
  - A variável para um objeto classe tem uma referência válida
- No entanto, existem casos particulares que podem produzir erro durante a execução
- Um programa que considere qualquer tipo de situação, normal ou anormal, torna-se complicado de escrever, de interpretar, de intervir ou de manter (excesso de *if-elses*)
- O mecanismo de exceções do Java permite separar a lógica normal da aplicação da lógica de tratamento de situações excecionais
- Durante a execução do programa, na presença de um evento extraordinário, é lançada uma exceção
  - Divisão por 0 (`ArithmeticException`)
  - Referência a *null* (`NullPointerException`)
  - Acesso a um *array* com um índice inválido (`ArrayIndexOutOfBoundsException`)

# Trio *try-throw-catch*

- O trio try-throw-catch é usado para lidar com exceções

```
try {  
    <Código com a lógica normal  
    de processamento>  
} catch (<ExceptionType> e) {  
    <Código com a lógica de  
    tratamento da exceção>  
}  
<Bloco de código a seguir  
a bloco try-catch>
```

Em qualquer ponto do bloco pode ser lançada uma exceção (throw). A exceção pode ser lançada por qualquer método chamado dentro do bloco

No caso de ser lançada uma exceção no bloco try, a execução do programa é transferida de imediato para o bloco catch que sirva a exceção. Pode ser definido mais do que um bloco catch para tratar de forma diferente diferentes tipos de exceção. A variável e guarda a referência para um objeto exceção

Este bloco de código é executado a seguir ao bloco try, se não foi gerada qualquer exceção (o bloco catch não é executado), ou após a execução do bloco catch, desde que não provoquem o retorno do método ou a terminação do programa antes de terminarem o bloco

# Exemplo try-catch

```
public static void tryCatchDemo() {  
    Scanner in = new Scanner( System.in ); int a, b; boolean finish;  
    try { do {  
        System.out.println("Terminar? (Y/n)");  
        finish = in.next().charAt(0) == 'Y';  
        if ( !finish ) {  
            System.out.print("Op A: "); a = in.nextInt();  
            System.out.print("Op B: "); b = in.nextInt();  
            System.out.println(a + " / " + b + " = " + (a / b));  
            System.out.println(a + " * " + b + " = " + (a * b));  
        } } while( !finish );  
    } catch ( Exception e ) { System.out.println(e.getMessage()); }  
    System.out.println("Termina");  
}
```

# Exemplo – sem exceção

```
public static void tryCatchDemo() {  
    Scanner in = new Scanner( System.in ); int a, b; boolean finish;  
    try { do {  
        System.out.println("Terminar? (Y/n)");  
        finish = in.next().charAt(0) == 'Y';  
        if ( !finish ) {  
            System.out.print("Op A: "); a = in.nextInt();  
            System.out.print("Op B: "); b = in.nextInt();  
            System.out.println(a + " / " + b + " = " + (a / b));  
            System.out.println(a + " * " + b + " = " + (a * b));  
        } } while( !finish );  
    } catch ( Exception e ) { System.out.println(e.getMessage()); }  
    System.out.println("Termina");  
}
```

```
Terminar? (Y/n)  
n  
Op A: 10  
Op B: 2  
10 / 2 = 5  
10 * 2 = 20  
Terminar? (Y/n)  
Y  
Termina
```

Quando o utilizador termina os cálculos (sai do bloco try), o programa continua a executar a seguir ao bloco catch

# Exemplo – com exceção `ArithmeticException`

```
public static void tryCatchDemo() {  
    Scanner in = new Scanner( System.in ); int a, b; boolean finish;  
    try { do {  
        System.out.println("Terminar? (Y/n)");  
        finish = in.next().charAt(0) == 'Y';  
        if ( !finish ) {  
            System.out.print("Op A: "); a = in.nextInt();  
            System.out.print("Op B: "); b = in.nextInt();  
            System.out.println(a + " / " + b + " = " + (a / b));  
            System.out.println(a + " * " + b + " = " + (a * b));  
        } } while( !finish );  
    } catch ( Exception e ) { System.out.println(e.getMessage()); }  
    System.out.println("Termina");  
}
```

```
Terminar? (Y/n)  
n  
Op A: 10  
Op B: 0  
/ by zero  
Termina
```

O bloco try termina logo que é gerada exceção, executa o bloco catch (afixa no *standard output* mensagem alusiva à exceção) e continua a executar a seguir ao bloco catch

# Exemplo – com exceção InputMismatchException

```
public static void tryCatchDemo() {  
    Scanner in = new Scanner( System.in ); int a, b; boolean finish;  
    try { do {  
        System.out.println("Terminar? (Y/n)");  
        finish = in.next().charAt(0) == 'Y';  
        if ( !finish ) {  
            System.out.print("Op A: "); a = in.nextInt();  
            System.out.print("Op B: "); b = in.nextInt();  
            System.out.println(a + " / " + b + " = " + (a / b));  
            System.out.println(a + " * " + b + " = " + (a * b));  
        } } while( !finish );  
    } catch ( Exception e ) { System.out.println(e.getMessage()); }  
    System.out.println("Termina");  
}
```

```
Terminar? (Y/n)  
n  
Op A: x  
null  
Termina
```

O bloco try termina logo que é gerada exceção. A exceção foi gerada no método nextInt por não se ter introduzido um inteiro. Nesta exceção, a mensagem alusiva ao erro está a null. Continua a executar a seguir ao bloco catch

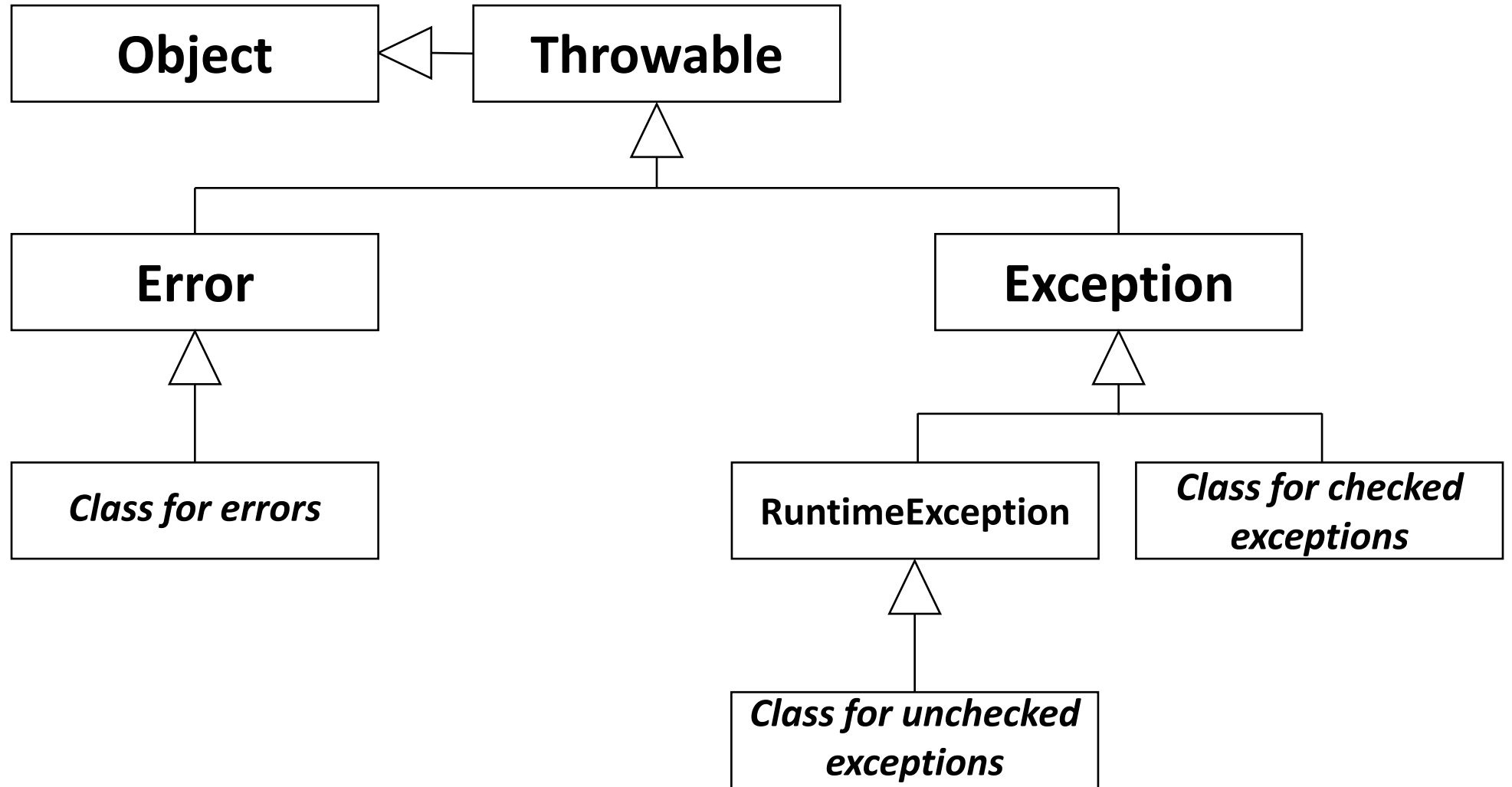
# Observações

---

- Factos observados com as demonstrações anteriores
  - A execução do bloco *try* é interrompida logo que ocorre exceção
    - Note-se que a exceção pode ser gerada ao nível do bloco *try* (a/b) mas pode igualmente ser gerada numa cadeia de chamadas dentro do bloco *try* (`in.nextInt()`)
  - O bloco *catch* é executado apenas na ocorrência de uma exceção compatível com a exceção capturada
  - Podem ser gerados diferentes tipos de exceções
  - O programa continua a executar após o bloco *try-catch*
- Questões em aberto
  - Como se dá um tratamento diferenciado para cada tipo de exceção?
  - De onde vem a mensagem alusiva à exceção?
  - É possível lançar explicitamente uma exceção?
  - O que se deve fazer quando não se sabe o que fazer em caso de exceção?
  - É possível criar um novo tipo exceção?
  - Alguma forma de concentrar código comum ao bloco *try* e ao bloco *catch*?

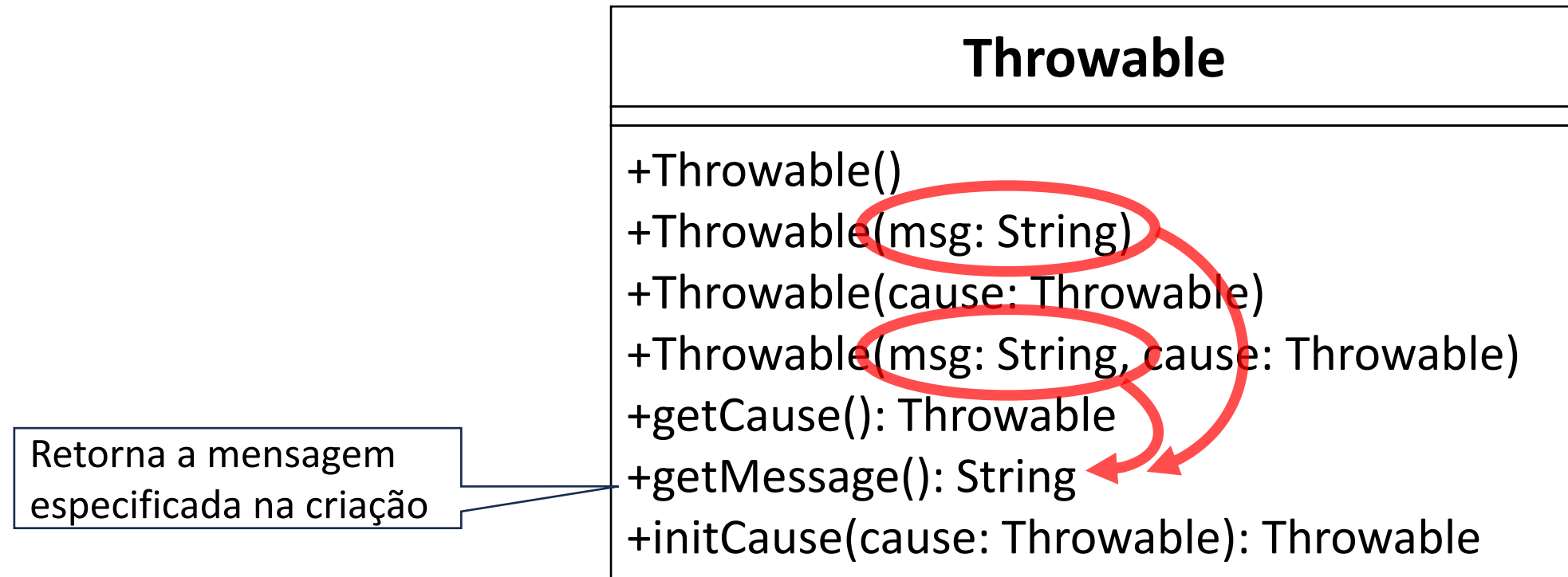
# Hierarquia de classes exceção predefinidas

---



# UML da classe Throwable

---



# Tipos de exceções (1 de 2)

---

- Tipo `Throwable`

- Superclasse de todos os erros e exceções
- Apenas objetos deste tipo ou derivados são lançados pela Máquina Virtual Java, são lançados explicitamente no programa ou aparecem na cláusula *catch*
- Reúne estado comum: mensagem com informação do erro; causa; *stack frames* de todos métodos chamados até ao método onde foi gerada a exceção

- Tipo `Error`

- Gerados quando, tipicamente, ocorrem situações anormais no sistema (ex: `OutOfMemoryError`)
- Por norma, não são tratáveis e o programa termina
- Não precisam ser capturados (cláusula *catch*) nem declarados na assinatura do método
- Tecnicamente, não são considerados exceções (embora sejam similares em tudo)

# Tipos de exceções (2 de 2)

---

- Todas as exceções em Java são classificadas como *exceções verificadas* (*checked*) ou como *exceções não verificadas* (*unchecked*)
- Tipo `Exception`
  - Todas as classes derivadas com um ou mais níveis são exceções verificadas (ex: `IOException`, `NoSuchMethodException`, ...)
  - Sendo exceções verificadas, têm de ser capturadas ou declaradas na assinatura do método
  - Representam uma situação anormal no programa que deve ser tratada (bloco *catch*)
- Tipo `RuntimeException`
  - Todas as classes derivadas com um ou mais níveis são exceções não verificadas (ex: `ArrayIndexOutOfBoundsException`, `NullPointerException`, ...)
  - Sendo exceções não verificadas, não têm de ser capturadas ou declaradas na assinatura do método
  - Representam um erro no programa que deve ser corrigido

# Blocos *catch*

---

- Num bloco *try-catch* pode existir mais do que um bloco *catch*

```
try {  
    <try block code>  
} catch (Exception1 e) {  
    <catch block code 1>  
} catch (Exception2 e) {  
    <catch block code 2>  
}
```

- A JVM procura pela primeira cláusula *catch* (*Exception1*, *Exception2*, por esta ordem) que sirva o tipo de objeto exceção gerado
- Executa apenas um único bloco *catch*
- No caso de se usar uma hierarquia de exceções relacionadas por herança, deve-se primeiro capturar a exceção mais especializada e, em segundo, a exceção mais geral (por exemplo, a classe *Exception* não deverá ser a primeira quando existirem vários blocos *catch*)

# Exemplo – blocos *catch*

O compilador dá erro indicando que a exceção `ArrayIndexOutOfBoundsException` já foi capturada porque deriva de `Exception`

```
public static void moreThanOneCatchWrongDemo() {  
    int[] a = {0, 1, 2, 3}; Scanner scanner = new Scanner(System.in);  
    try { System.out.println("Insira um inteiro:"); a[3] = scanner.nextInt();  
        System.out.println("Insira outro inteiro:"); a[4] = scanner.nextInt();  
    } catch (Exception e) { System.out.println("1: " + e.getClass() + ", " + e.getMessage()); }  
    catch (ArrayIndexOutOfBoundsException e) { System.out.println("2: "+e.getClass()+" "+e.getMessage()); }  
}
```

```
public static void moreThanOneCatchRightDemo() {  
    int[] a = {0, 1, 2, 3}; Scanner scanner = new Scanner(System.in);  
    try { System.out.println("Insira um inteiro:"); a[3] = scanner.nextInt();  
        System.out.println("Insira outro inteiro:"); a[4] = scanner.nextInt();  
    } catch (ArrayIndexOutOfBoundsException e) { System.out.println("1: "+e.getClass()+" "+e.getMessage()); }  
    catch (Exception e) { System.out.println("2: " + e.getClass() + ", " + e.getMessage()); }  
}
```

# Lançar uma exceção

---

- O programador pode lançar explicitamente uma exceção
- Tem de criar um objeto do tipo da exceção que pretende lançar
- Por norma, está enquadrada numa instrução `if`  
`if (abnormal_condition)`  
`throw new <ExceptionType>(arguments_list);`
- Transfere de imediato a execução para o primeiro bloco *catch* que capture a exceção lançada dentro do *stack* de chamadas
  - Pode ser `<ExceptionType>` ou superclasse
  - Termina o programa se a exceção não for capturada no *stack* de chamadas
- Ex:  
`throw new Exception("Some exception description");`  
`throw new FileNotFoundException();`
- A mensagem especificada na criação da exceção pode ser obtida no bloco *catch* com o método `getMessage()`

# Declaração de exceções na assinatura de um método

---

- Uma exceção verificada (*checked exception*) precisa ser capturada
- Se a exceção não for capturada no método onde é lançada, o método tem de declarar na assinatura que pode terminar por exceção

```
type method(args) throws exception1, exception2, ... {  
    /* body */  
}
```
- Tem de declarar todas as exceções verificadas que não são capturadas no método
- No *stack* de chamadas, quaisquer exceções não capturadas têm de estar declaradas nos respetivos métodos
- Um método que sobrepõe outro método da classe base não pode acrescentar novas exceções na cláusula *throws* (a menos que sejam especializações); apenas pode suprimir (porque as captura)

# Definição de uma nova exceção

---

- Um novo tipo de exceção tem de derivar de alguma exceção previamente definida (tipicamente, deriva de `Exception`)
- Uma nova exceção define, por norma, dois construtores
  - Construtor sem parâmetros
  - Construtor com um parâmetro do tipo `String`
- O construtor sem parâmetros chama o construtor da classe base passando-lhe como argumento uma *string* com uma mensagem, por omissão
- O construtor com o parâmetro *string* deve chamar o construtor da classe base passando-lhe como argumento a *string* recebida como parâmetro
- Recordar-se que a *string* especificada na criação da exceção é retornada pelo método `getMessage` definido em `Throwable` e herdado nas classes derivadas (portanto, para cumprir este objetivo, o método `getMessage` não deve ser redefinido)
- A definição de uma nova exceção deve preservar este comportamento

# Exemplo – definição de uma nova exceção, `throw` e `throws`

(1 de 3)

```
public class IllegalStockValueException extends Exception {  
    IllegalStockValueException() { super("Illegal stock value"); }  
    IllegalStockValueException(String msg) { super(msg); }  
}
```

*Checked exception*

# Exemplo – definição de uma nova exceção, throw e throws

## (2 de 3)

```
public class ThrowAndThrowsDemo {  
    public static final int STOCK_SIZE = 100, MAX_TRIES = 3;  
    static private Scanner in; static private int[] stock;  
    public static int readStockItem() throws IllegalArgumentException {  
        System.out.println("Insira valor entre 0 e 100:"); int v = in.nextInt();  
        if (v < 0 || v > 100)  
            throw new IllegalArgumentException("Invalid stock value: entered " + v + ", range valid: 0..100");  
        return v;  
    }  
    public static void fillStock() throws IllegalArgumentException {  
        for (int i = 0; i < stock.length; i++)  
            stock[i] = readStockItem();  
    }  
    // ...  
}
```

Checked exception. Como não é capturada tem de ser declarada no método

Checked exception. Como não é capturada tem de ser declarada no método

# Exemplo – definição de uma nova exceção, throw e throws

(3 de 3)

```
public class ThrowAndThrowsDemo {  
    // ...  
    public static void main(String[] args) {  
        in = new Scanner(System.in); stock = new int[STOCK_SIZE];  
        int nTries = MAX_TRIES; boolean success = false;  
        while (nTries > 0 && !success)  
            try { fillStock(); success = true; }  
            catch (IllegalStockValueException e) {  
                System.out.println(e.getMessage());  
                if (nTries > 0) System.out.println("Fill again");  
                nTries -= 1;  
            }  
    }  
}
```

Dá MAX\_TRIES tentativas para preencher o array stock

Devolve a mensagem usada na criação da exceção

```
Insira valor entre 0 e 100:  
-1  
Invalid stock value:  
entered -1, range valid:  
0..100  
Fill again  
Insira valor entre 0 e 100:  
...
```

# Bloco *finally*

---

- O bloco *finally* é opcional e pode ser acrescentado a seguir a um bloco *try-catch*

```
try { <try block code> }  
catch (Exception1 e) { <catch block code> }  
finally { <finally block code> }
```

- Garante a execução do código do bloco *finally* para qualquer cenário de execução do bloco *try-catch* que o antecede
  - O bloco *try* executa até ao fim sem que tenha sido gerada qualquer exceção  $\Rightarrow$  o bloco *finally* é executado logo a seguir
  - Durante a execução do bloco *try* é gerada uma exceção e capturada por um bloco *catch*  $\Rightarrow$  depois de executado o bloco *catch*, é executado o bloco *finally*
  - Durante a execução do bloco *try* é gerada uma exceção que não é capturada por nenhum dos blocos *catch* e, por isso, o método onde ocorreu a exceção vai retornar de imediato  $\Rightarrow$  o bloco *finally* é executado antes do método retornar

## Exemplo – bloco *finally* (1 de 2)

```
public class AException extends Exception {  
    AException() { super("AException"); }  
}
```

```
public class BException extends Exception {  
    BException() { super("BException"); }  
}
```

```
public class CException extends Exception {  
    CException() { super("CException"); }  
}
```

```
public class ExceptionsGenerator {  
    static Scanner in = new Scanner(System.in);  
  
    static String generate() throws  
        AException, BException, CException {  
        String s = in.next();  
        switch(s.charAt(0)) {  
            case 'a','A': throw new AException();  
            case 'b','B': throw new BException();  
            case 'c','C': throw new CException();  
        }  
        return s;  
    }  
}
```

## Exemplo – bloco *finally* (2 de 2)

```
public class FinallyDemo {  
    public static void main(String[] args) throws CException {  
        int cnt = 3;  
        while (cnt-- > 0) {  
            System.out.println("Insira palavra");  
            try { System.out.println(ExceptionsGenerator.generate()); }  
            catch (AException e) { System.out.println("1: " + e.getMessage()); }  
            catch (BException e) { System.out.println("2: " + e.getMessage()); }  
            finally { System.out.println("3: Finally block"); }  
            System.out.println("Doing again");  
        }  
    }  
}
```

Insira palavra  
ab  
1: AException  
3: Finally block  
Doing again

Insira palavra  
de  
de  
3: Finally block  
Doing again

Insira palavra  
cd  
3: Finally block  
Exception in thread "main" CException: CException

# Blocos *try-catch* aninhados

---

- Raramente utilizado e, por norma, revelador de má organização do código
- No entanto é possível
  - Bloco *try-catch* aninhado dentro de um bloco *try*
  - Bloco *try-catch* aninhado dentro de um bloco *catch*
- Uma exceção lançada dentro do bloco *try-catch* aninhado pode ser capturada
  1. Pelas cláusulas *catch* do bloco aninhado
  2. Pelas cláusulas *catch* do bloco *try-catch* de fora, se nenhuma cláusula *catch* do bloco aninhado servir o tipo de exceção lançada

# Lançar uma exceção dentro do bloco *catch*

---

- Válido lançar uma exceção dentro do bloco *catch*
- Em raros casos, pode-se pretender capturar uma exceção, realizar algum tratamento e lançar novamente a mesma ou outra exceção para ter tratada acima na cadeia de tratamento de exceções
- Outro exemplo é em redefinições de métodos em classes derivadas
  - Na sobreposição de um método não se pode acrescentar novas exceções na cláusula *throws*
  - Durante a execução do método, pode ser lançada uma exceção não prevista e, portanto, não declarada no método da classe base
  - A solução passa por capturar a nova exceção no método redefinido e, eventualmente, criar um objeto exceção previsto no método da classe base, estabelecer como causa a nova exceção, e relançar

# Exemplo – exceções aninhadas e relançamento (1 de 3)

```
public class AException extends Exception {  
    AException() { super("AException"); }  
    AException(String msg) { super(msg); }  
}
```

```
public class BException extends Exception {  
    BException() { super("BException"); }  
    BException(String msg) { super(msg); }  
}
```

```
public class CException extends Exception {  
    CException() { super("CException"); }  
    CException(String msg) { super(msg); }  
}
```

```
public class ExceptionsGenerator {  
    static Scanner in = new Scanner(System.in);  
  
    static String generate() throws  
        AException, BException, CException {  
        String s = in.next();  
        switch(s.charAt(0)) {  
            case 'a','A': throw new AException();  
            case 'b','B': throw new BException();  
            case 'c','C': throw new CException();  
        }  
        return s;  
    }  
}
```

## Exemplo – exceções aninhadas e relançamento (2 de 3)

```
public class NestedExceptionsDemo {  
    public static void main(String[] args) throws BException, CException {  
        int cnt = 10; System.out.println("Insira palavras");  
        while (cnt-- > 0)  
            try {  
                try { System.out.println(ExceptionsGenerator.generate()); }  
                catch(AException e) { System.out.println("Inner try: " + e.getMessage()); }  
            }  
            catch(BException e) {  
                System.out.println("Outer: " + e.getMessage());  
                try { System.out.println(ExceptionsGenerator.generate()); }  
                catch(AException e1) { System.out.println("Inner catch: " + e1.getMessage()); }  
            }  
            catch(CException e) { System.out.println("Outer: " + e.getMessage()); }  
    }  
}
```

## Exemplo – exceções aninhadas e relançamento (3 de 3)

---

Insira palavras

abc

Inner try: AException

cde

Outer: CException

bcd

Outer: BException

abc

Inner catch: AException

bcd

Outer: BException

bcd

Exception in thread "main" BException: BException

at NestedExceptionsDemo.exceptionsGenerator(NestedExceptionsDemo.java:11)

at NestedExceptionsDemo.main(NestedExceptionsDemo.java:27)